



Chapter 5 Virtual Memory

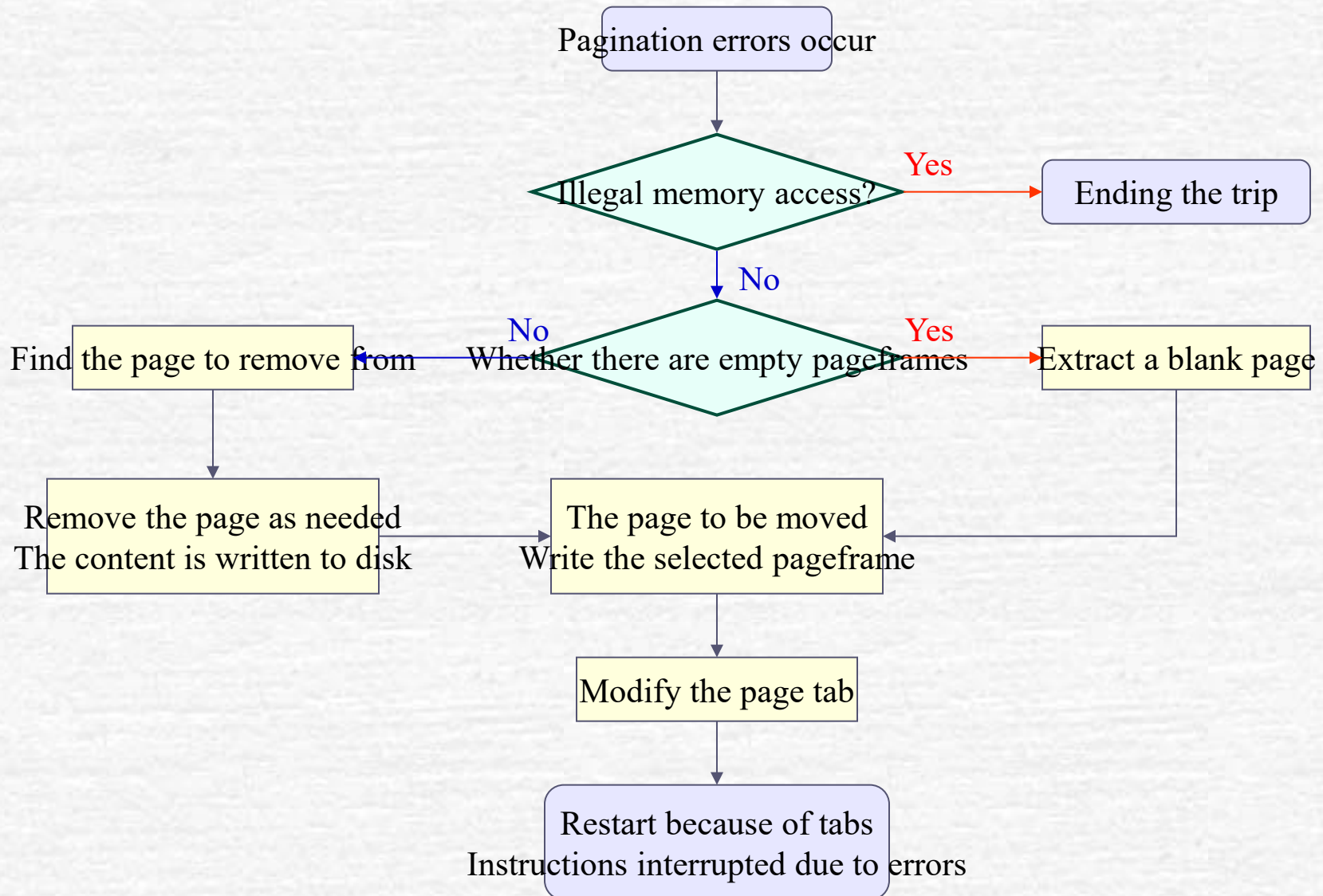
5-1 Basic Concepts

- The disadvantage of putting the entire program into physical memory :
 - During process execution, not all parts of the program are used
 - Programmers must carefully consider the length of the program to avoid exceeding the limits of physical memory

Basic concepts

- **Virtual memory** : A contiguous and massive logical address space allows programmers to write programs without worrying about length limitations
 - Using demand pagination, pages that are not currently used by the process are "packaged" into auxiliary memory
- **Paging program** : The program responsible for moving pages into and out of memory
 - Add a valid bit to each item in the page table, which is checked by the memory management hardware
- **Pure Needs Pagination** : When running the program at first, no pages are stored in memory at all

Fig5-1 Handling of pagination errors



Pagination and Replacement Programs

- **substitution program** : It handles the permutation between main and auxiliary memory throughout the process. The swapped process enters the swap state and is not considered for short-term scheduling
- **Paging program** : Under the requirements pagination mechanism, it is responsible for moving process pages between main memory and auxiliary memory

Operational considerations for virtual memory

- We hope to allow more processes to enter memory and execute
 - How many page frames should be allocated to each process?
 - Do you want to set a fixed or variable number of frames?
- You want to keep pages frequently or recently used by each process in main memory to improve virtual memory access efficiency
- Design of page replacement operations

Pagination error handling time

- Memory access time: between 10ns and 200ns
- Assuming the memory access time is m , and no paging errors occur, the time to read the required word from memory is m
- If a pagination error occurs, the process must first wait for the operating system to load the required page
- Assuming the time to handle the paging error is t , the word's access time becomes $t+m$
- Factors affecting t : handling control transfers caused by interrupts, searching pageframes, performing disk reads, and restarting execution, all take about 25ms
- Assuming $m = 25\text{ns}$, the time ratio for main memory access to auxiliary memory access is approximately 1:1,000,000
- If there is one pagination error per 1000 memory accesses: $t+m = 25000000+25$
- The average access time becomes about 25 microseconds: $(25,000,000 + 25 + 25*999) / 1000$
- That is, 1000 times the simple memory access

5-2 Page replacement

- Principle of locality
- Page replacement algorithm
- Choosing the page size

Principle of locality

- **Principle of locality** : Memory access performed by a process at a given time is usually concentrated around several major addresses, also known as **locality of references**
- The reason for the formation of the principle of locality :
 - When running a program, unless it encounters a branch or function call, it always runs sequentially. Therefore, it is mostly accessed by contiguous instruction addresses
 - If a program executes instructions like loops, it will repeatedly iterate between small segments of code
 - The large data structures used in programs usually employ contiguous memory addresses

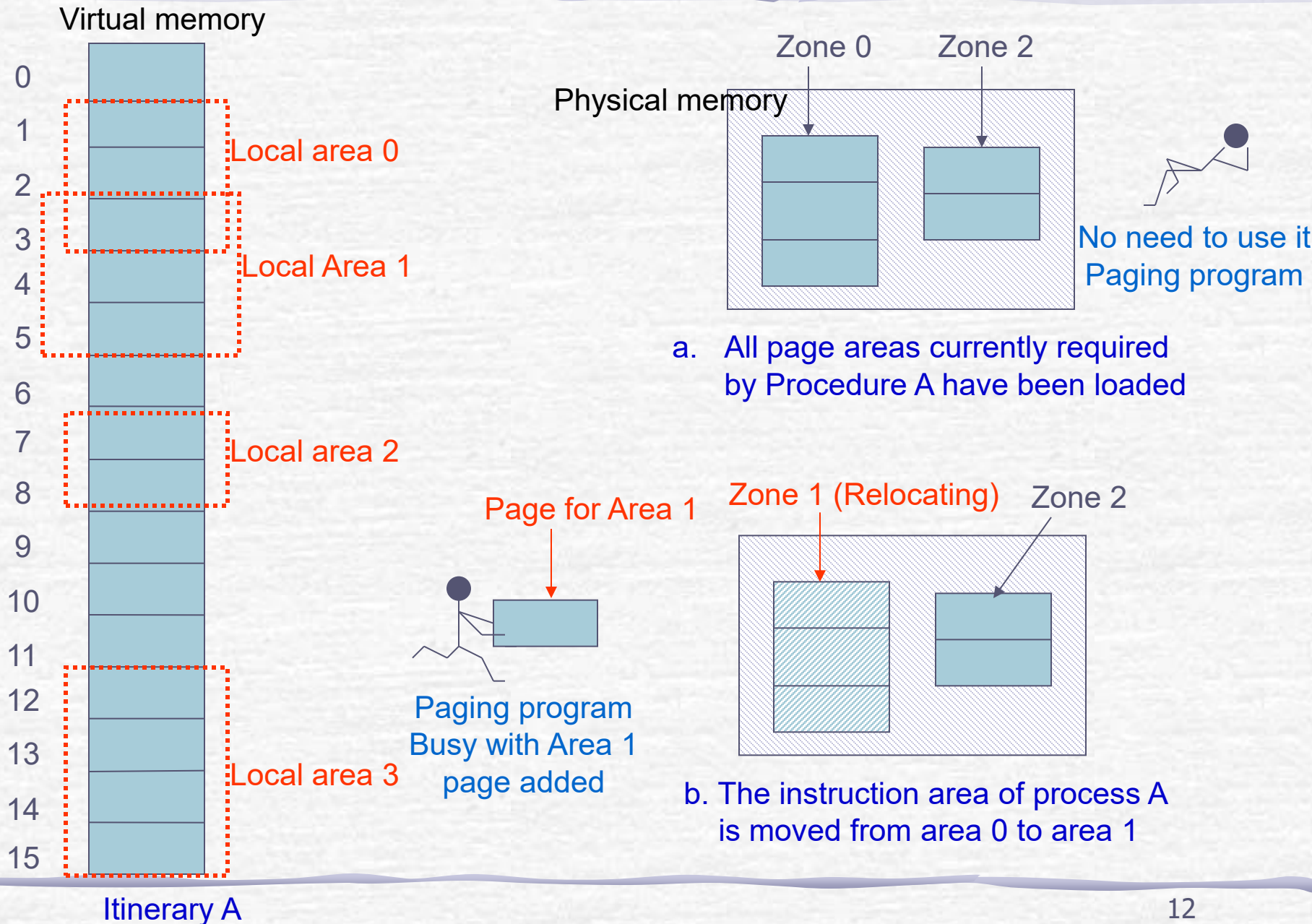
A localized type

- **The locality of time** : If a memory location has just been accessed, it may soon be accessed again; for example, the memory location storing the main variable in a program is likely to be accessed frequently
- **The locality of space** : Once a memory location has been accessed, nearby locations may be accessed quickly, such as sequential instructions or array data

View process execution from a local area perspective

- **Local areas** : A set of pages that will be used simultaneously within a certain period
- A program may contain many different local areas, and these local areas may overlap with each other
- The actual execution order of a process is like moving between different local areas

Figure 5-2 View of process execution from a local area perspective



Page replacement algorithm

- **Optimal replacement algorithm** : Ideally, it's best to remove pages that will never be used again; the worst option is to remove pages that will only be used in the future
 - It can only be used as a benchmark for comparison and cannot be implemented
 - Example :

Reference page number		1	2	1	3	5	1	0	5	2	1	5	1
page frame	{		1	1	1	1	1	0	0	0	1	1	1
				2	2	2	2	2	2	2	2	2	2
					3	5	5	5	5	5	5	5	5

There were a total of 6 pagination errors

FIFO algorithm

- The most intuitive way is to replace the pages that have stayed in memory the longest and replace them with new page content
- Fields with load times must be included in the pages of the page table
- It does not consider page usage, which may cause frequently used pages to be moved out and in, leading to overall performance deterioration

Reference page number		1	2	1	3	5	1	0	5	2	1	5	1
page frame		1	1	1	1	5	5	5	5	2	2	2	2
			2	2	2	2	1	1	1	1	1	5	5
					3	3	3	0	0	0	0	0	1

There were a total of 9 pagination errors

LRU algorithm

- Move out pages that haven't been used the longest
- According to the principle of locality, if a page has only recently been used, it may be used again in the near future
- It is a very effective page replacement algorithm, but because it needs to track the last access time, its implementation is more complex
 - The simplest approach is to add a counter to the memory management hardware that automatically increments by 1 whenever a memory access occurs
 - For each item in the paged table, add one field to store the counter's value
 - When accessing a page, the counter value for that time is stored in the corresponding page entry in the page tab
 - When pages need to be replaced, the LRU algorithm will identify the page with the lowest field value to replace it

Figure 5-5 LRU algorithm example

Reference page number		1	2	1	3	5	1	0	5	2	1	5	1
page frame		1	1	1	1	1	1	1	1	2	2	2	2
			2	2	2	5	5	5	5	5	5	5	5
					3	3	3	0	0	0	1	1	1

There were a total of 7 pagination errors

LRU Approximation Method - Aging algorithm

- The aging algorithm adds a reference byte to each page frame in memory
- When a page is accessed, the hardware sets the most important bit in the byte to 1 and uses a timer interrupt to periodically shift the entire reference byte to the right by 1 bit
- When page replacement is needed, the page with the smallest binary value in the reference byte is replaced

Figure 5-6 Example of aging algorithm

Reference bytes	Main memory
00000000	Page 18
00000000	Page 9
00000000	Page 29
00000000	Page 3

(a) Initial Status

Reference bytes	Main memory
10000000	Page 18
00000000	Page 9
10000000	Page 29
00000000	Page 3

(b) Pages 18 and 29 were accessed

01000000	Page 18
00000000	Page 9
01000000	Page 29
00000000	Page 3

(c) The timer interrupts, and the bit shifts

11000000	Page 18
10000000	Page 9
01000000	Page 29
10000000	Page 3

(d) Pages 3, 9, and 18 are accessed

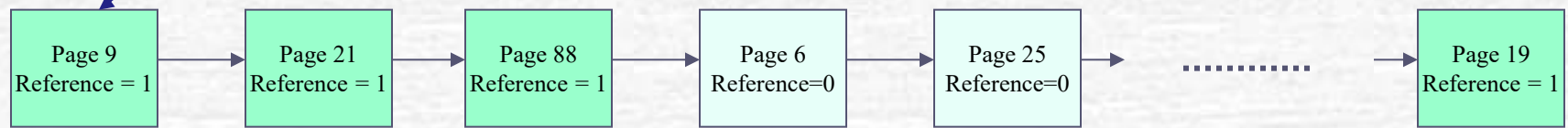
LRU Approximation Method - Quadratic Chance Algorithm

- Each memory page frame corresponds to only one reference bit, and its initial value is 0
- When a page is accessed, its corresponding reference bit is set to 1
- When the system needs to replace pages, it first searches for the pages that have stayed in memory the longest in the order of the FIFOs
 - If the reference bit for that page is 0, it will be replaced by a new page
 - If its reference bit is 1, it gets a second chance: set its reference bit to 0, and its loading time is changed to the current time
 - Then continue searching for the next possible candidate page

Figure 5-7 Quadratic Chance Algorithm—Linked List

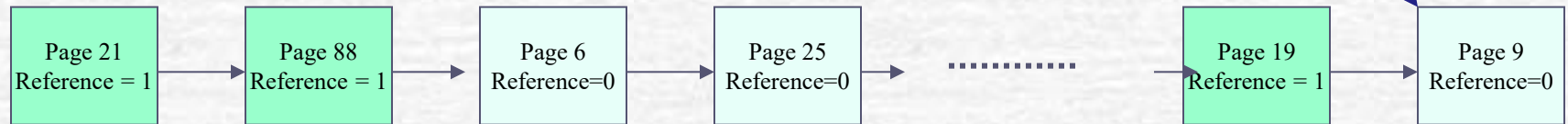
Page 75 A new page to add

The earliest page loaded

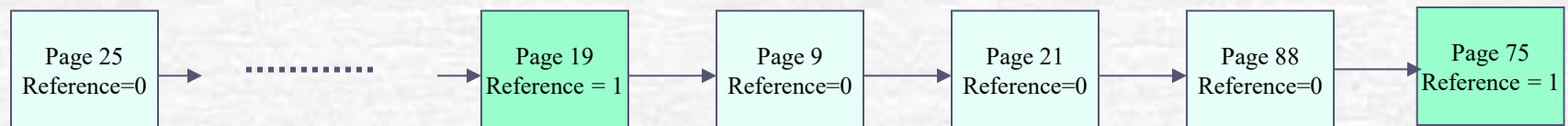


(a)

Treat page 9 as just loaded



(b)

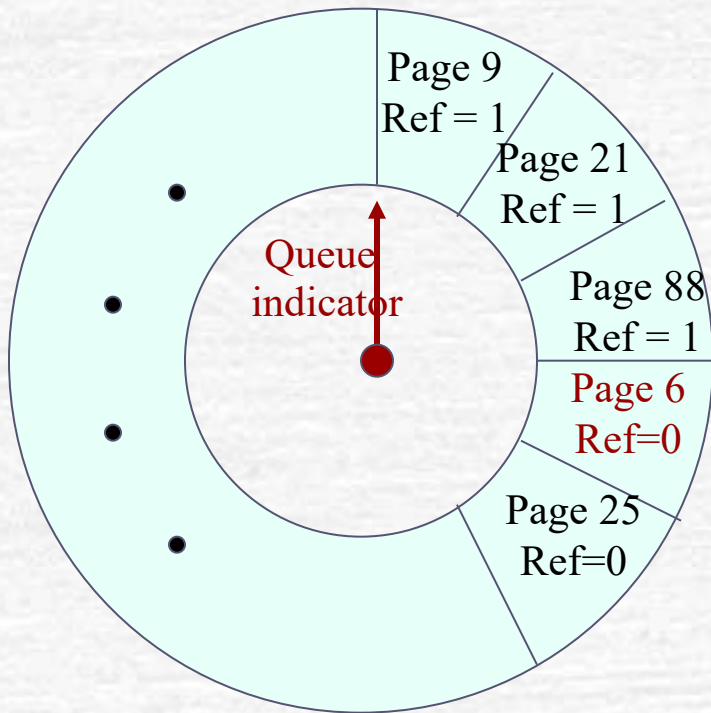


(c)

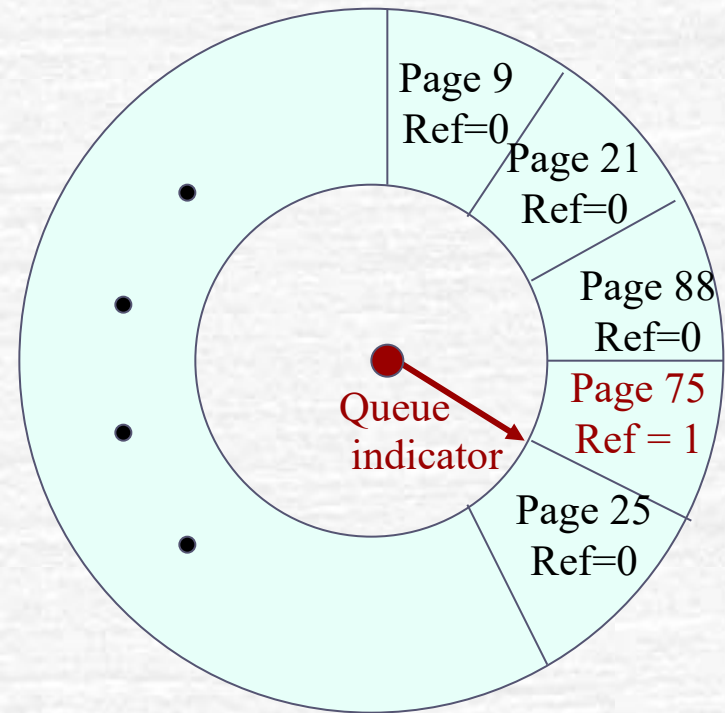
Implementation of the quadratic chance algorithm

- Implementing quadratic chances algorithms using linked lists is less efficient
- Another feasible approach is to implement a ring queue—also known as the clock algorithm
 - After each page replacement, move the queue pointer forward to the next page frame to save search time

Figure 5-8 Quadratic Chance Algorithm—Circular Queue



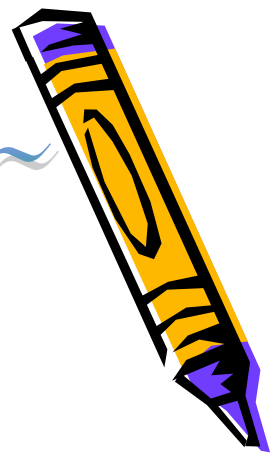
(a) Equivalent to Figure 5-7.a



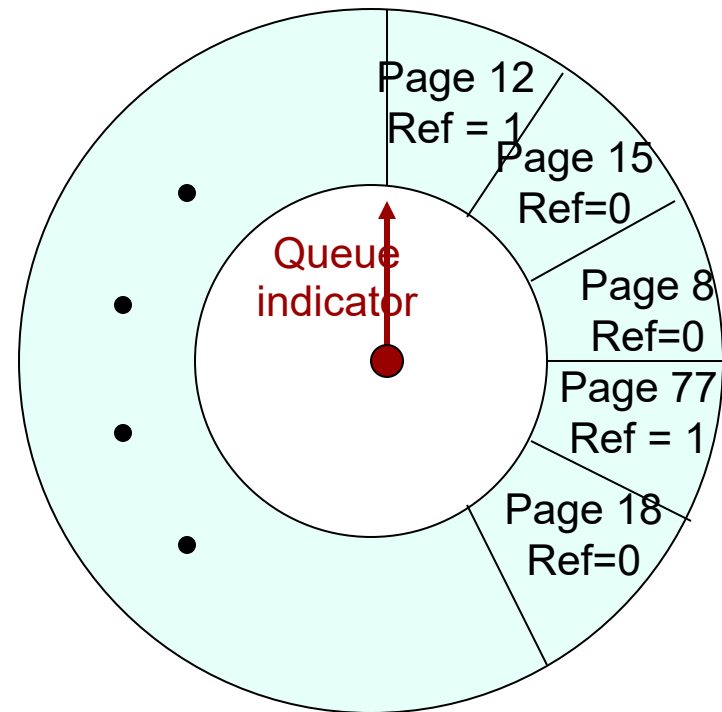
(b) Equivalent to Figure 5-7.c

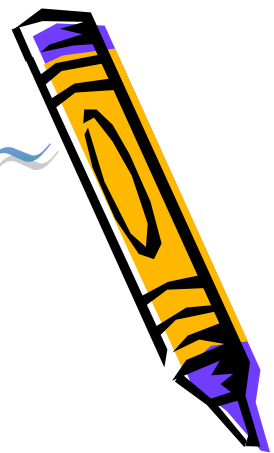
Classroom exercises

範例補充包-Extra~



- Suppose a quadratic algorithm is used for page replacement, and the current state of the circular queue is as follows. When new pages **22 and 33** are about to enter memory in order:
- Which two pages will be replaced?
- After the replacement is complete, what are the reference bit values for each page?
- Which page will the queue pointer point to?





Practice Answers

- Pages **15 and 8** will be replaced by pages **22 and 33** respectively
- The queue pointer points to **page 77**
- The new reference bit value is :

Page	12	22	33	77	18
Reference bit	0	1	1	1	0



Page locking

- While providing page replacement functionality, the operating system must also offer page locking features that lock specific pages in memory to prevent them from being moved to auxiliary memory
- Many parts of the operating system must be locked in memory, especially when **running page-in and page-out page-entry** pagination programs
- I/O systems also need to keep certain pages in memory so that data can be read or written directly from the process's memory locations
- To provide page locking, you can add a lock bit to the pages of the page table's items

Implementation Discussion - Windows Page Replacement Policy

- **In multiprocessor systems**, Windows uses locally ranged FIFO algorithms
- **In single-processor systems**, Windows uses quadratic algorithms implemented with a circular queue

Modify bits

- When performing pagination replacement, if the page has not changed, there is no need to write back to auxiliary memory
- To distinguish whether the replaced page has changed, you can add a modification bit, also called a dirty bit, to the page tab section
- The modification bit is initially cleared to zero, and when data is written to the page, the modification bit for that page is set

Page size

- The size of a page is determined by hardware design, which considers the following factors :
 - To match hardware access efficiency, pages are usually set to an exponent of 2
 - If the page length is small, the internal fragments will also be smaller
 - A larger page length means the page table occupies less memory space
 - When the page length is large, the efficiency of moving in and out is higher
 - The small page better reflects the locality of the itinerary
- Usually, the larger the process and physical memory, the larger the page must be

5-3 Page Frame Layout Strategy

- Each process contains n pages. Without the need pagination technique, m pages can only hold m/n pages
- However, since processes may not use all pages during execution, exploiting this feature to increase the number of multitasking tasks will result in memory over-allocation
- If a running process suddenly needs a large number of pages, it will cause memory shortages
- The solution :
 - Find the appropriate page to replace
 - Carefully decide the **number of page frames for each itinerary**
 - Maintain an appropriate level of multitasking

Rolling and turning phenomenon

- If the operating system allocated too few page frames for processes, a large number of page break errors and page moves may occur
- **Rolling Effect:** This phenomenon occurs when a process spends more time processing paging than executing instructions, resulting in a significant drop in performance
- If the number of page frames is **less than** the local area required for process execution, rollover may occur

Figure 5-9(a) Rolling phenomenon

Page 7 of process A is moved into the 10th frame of physical memory

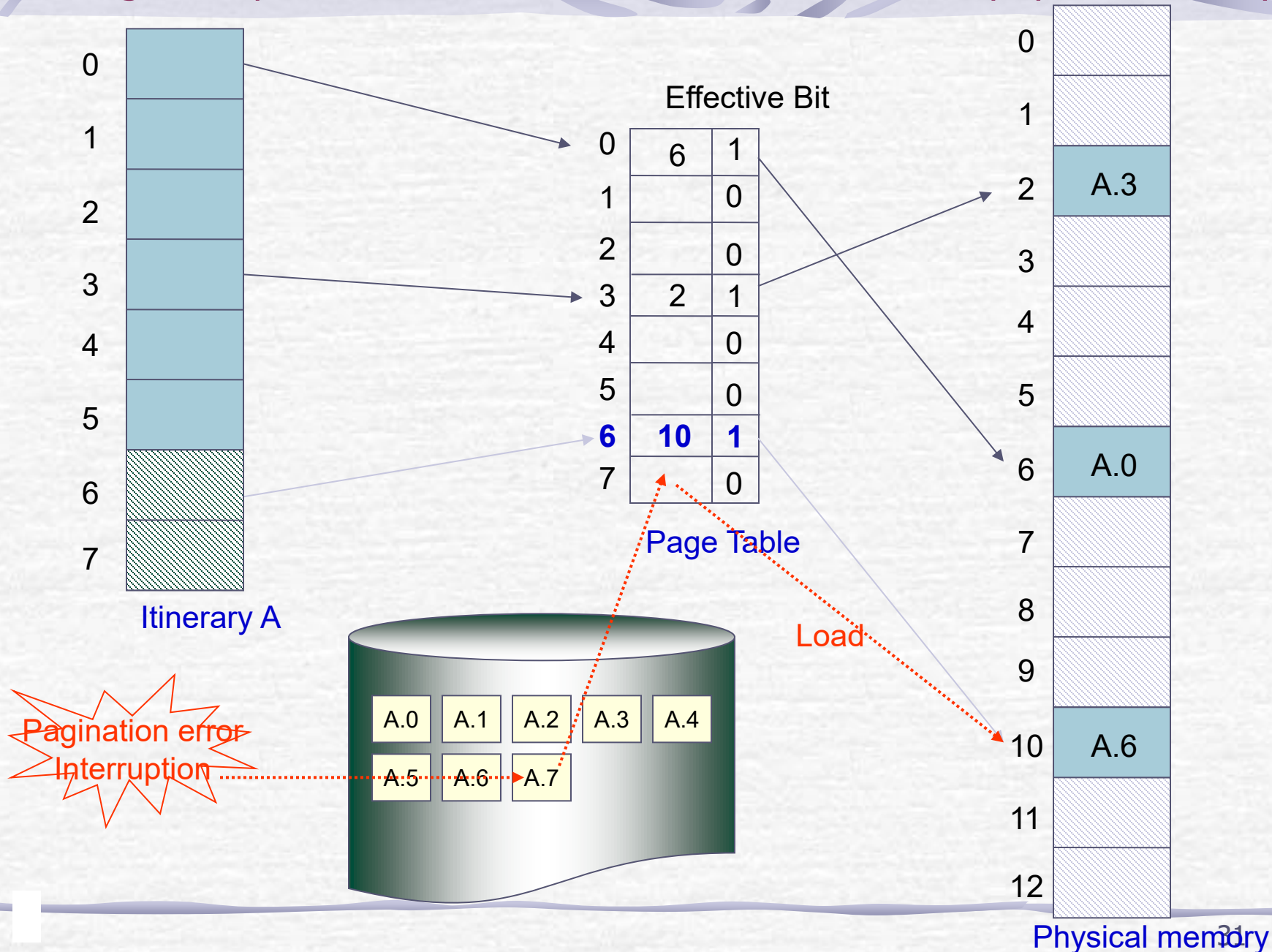
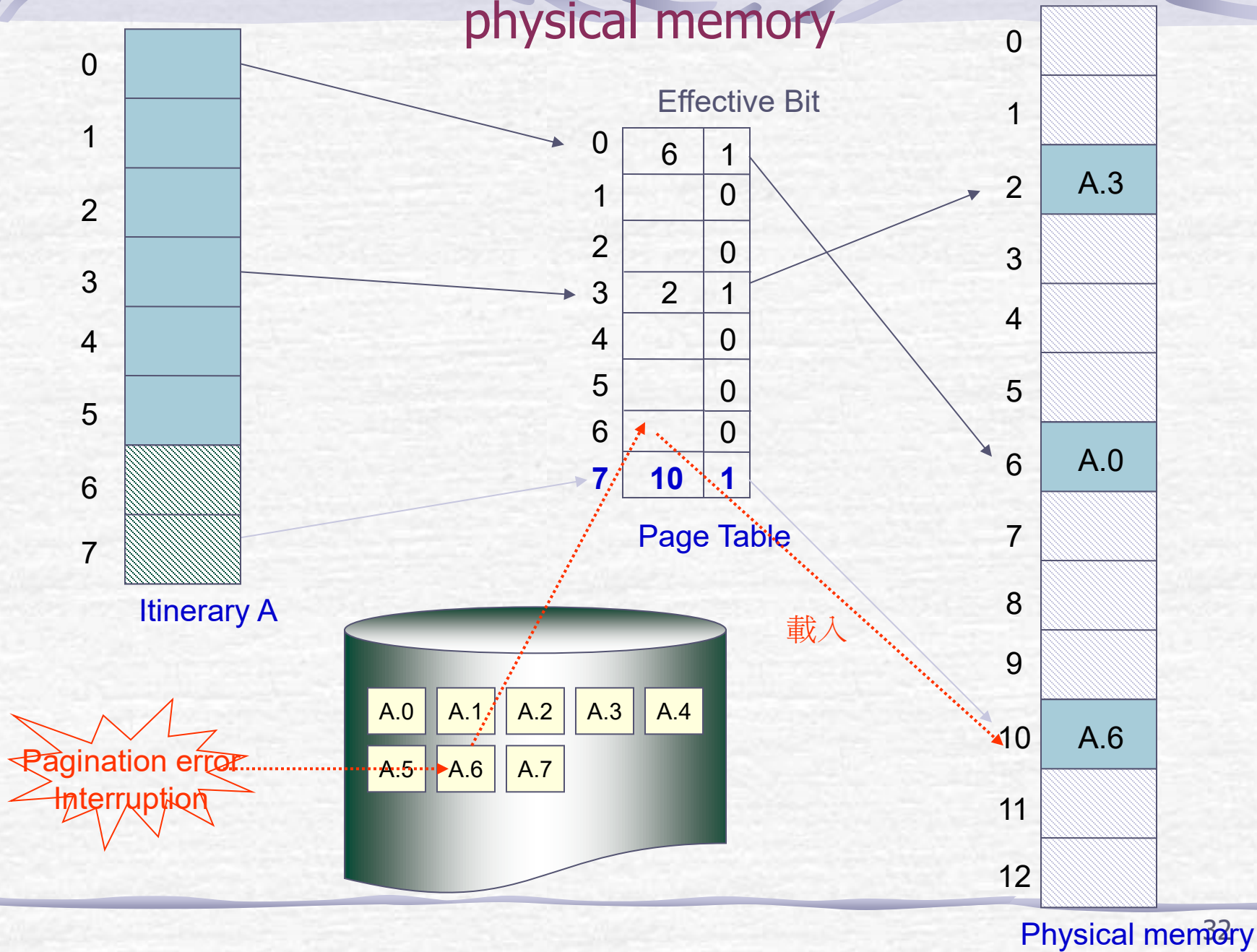


Figure 3-5(b) Relating Itinerary to Physical memory

Page 6 of process A is moved to the 10th frame of physical memory



Configuration of the number of page frames

- Considerations for the number of configurations :
 - Too few page frames increase the rate of page errors and may even cause the phenomenon of page rotation
 - However, to increase the system's multitasking capacity and improve resource utilization, it is not advisable to allocate too many pages to a single process
- **Belady anomaly** : If the algorithm's replacement selection is poor, the number of pages assigned to the process increases, but the number of page errors may increase

Figure 5-10 Example of Belady anomaly

Reference page number

		1	2	4	5	1	2	0	1	2	4	5	0
page frame		1	1	1	5	5	5	0	0	0	0	0	0
			2	2	2	1	1	1	1	1	4	4	4
				4	4	4	2	2	2	2	2	5	5

(a) Three page frames

There were a total of 9 pagination errors

Reference page number

		1	2	4	5	1	2	0	1	2	4	5	0
page frame		1	1	1	1	1	1	0	0	0	0	5	5
			2	2	2	2	2	2	1	1	1	1	0
				4	4	4	4	4	4	2	2	2	2
					5	5	5	5	5	5	4	4	4

(b) Four page frames

A total of 10 pagination errors occurred

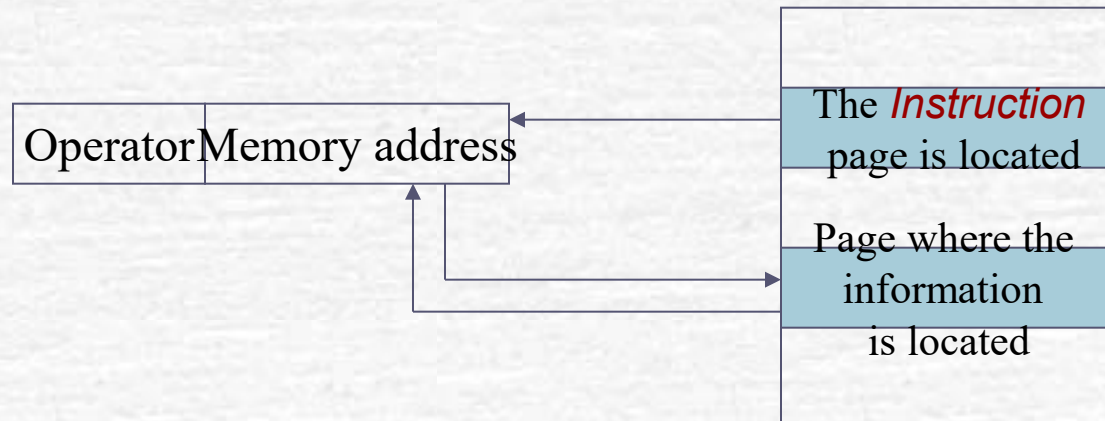
Page frame layout

- **Average configuration** : Assuming the number of memory frames is m and the total number of processes is n , then the number of frames each process can allocate is m/n
- **Proportional allocation** : Assume there are n processes in the current system , and the length of the i -th process is n_i , Then the number of page frames that the process can allocate is $m * n_i / \sum n_i$

Minimum configuration quantity

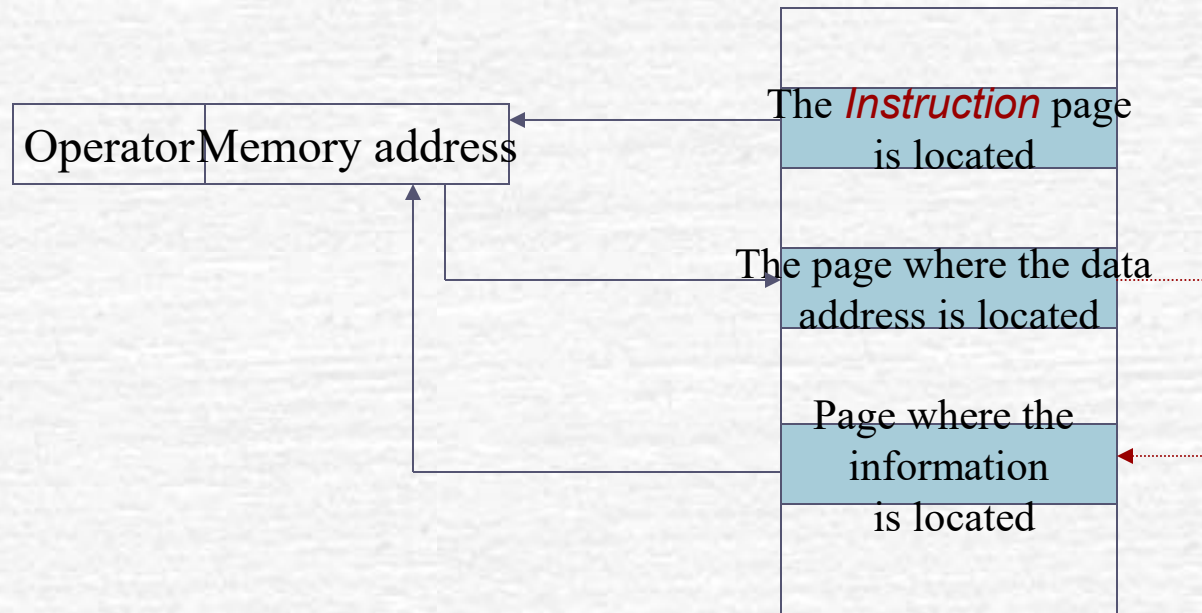
- All processes on the same system share a common minimum number of pageframes; This number is determined by the hardware CPU structure and instruction architecture
- **Instruction architecture :**
 - Direct addressing : At least 2 page frames are required (**Instruction + Data**)
 - Indirect addressing : At least 3 page frames are required (**Instruction + Data address + Data**)
- **CPU architecture :**
 - Systems with 2 operands: Assuming each operand can use indirect addressing, each process requires at least a minimum
 $1 + 2*2 = 5$ page frames
 - System with 3 operands: Assuming each operand can use indirect addressing, each process requires at least a minimum
 $1 + 2*3 = 7$ page frames

Figure 5-11 Example of minimum number of page frames



a. Direct addressing

At least two page frames are needed to store the pages needed for execution



b. 間接定址

至少需要3個頁框才夠存放執行所需的頁面

The scope of page replacement

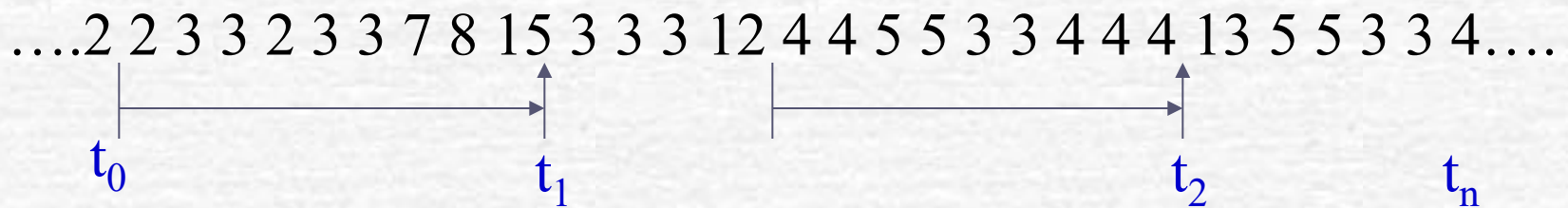
- **Fixed configuration strategy** : When a process is created, a fixed number of frames are pre-allocated according to its type
 - Localized page replacement
- **Change allocation strategies** : It allows you to change the number of page frames assigned to a process during execution
- **Page replacement across the entire scope** : The execution time of the process can vary greatly
 - Localized page replacement

Work set

- Work set $WS(t, \Delta)$: Represents the collection of pages referenced by the process within Δ time units calculated from time t to the process
- Characteristics of the work set
 - When Δ remains unchanged, the number of pages contained in the workset and the content of the pages inside change over time t
 - If t remains unchanged, as Δ grows, the number of pages included also increases; In other words, $WS(t, \Delta+1)$ will be greater than or equal to $WS(t, \Delta)$
- If an appropriately sized Δ can be found, the working set can be said to reflect the approximate situation of the local region

Figure 5-12 Paginated Access and Task Set

The page number referenced for each memory access :



$$WS(t_1, \Delta) = \{2, 3, 7, 8, 15\}$$

$$WS(t_2, \Delta) = \{3, 4, 5\}$$

Approximate local region 0 and local region 2

Approximate local area 1

Workset and page frame configuration

- Work set $WS(t, \Delta)$ The number of pages included is the total number of pages that will be used simultaneously during the Δ period
- Our goal is to find the appropriate Δ value
 - Δ value is too small : The work set may not contain the complete local area
 - Δ value is too great : It may also include pages from the previous area
- Once the appropriate Δ value is determined, , We can make use of the work set size of the $WS(t, \Delta)$, to decide how many frames to allocate to the process

Pre-paginated

- **Pre-paginated** : If the operating system has knowledge of the workset, it can load all pages in the workset in advance when processes are loaded or replaced into main memory
- Some loaded pages may not be used; These unused pages waste I/O bandwidth and memory frames
- The contribution of pre-pagination to system performance may be positive or negative, depending on the proportion of "**guesses correctly**."
- It can also be used in The **Pagination error** time

Pagination error frequency

- **Pagination error frequency** : This can more directly prevent the phenomenon of rolling over and over
 - Pagination error frequency is too high: the number of page frames configured may be insufficient
 - Pagination error frequency too low: Too many page frames may have been configured
- By setting upper and lower limits on the frequency of process pagination errors, the operating system can increase the number of allocated frames when the upper limit is exceeded, and reclaim some frames when it falls below the lower limit

Windows process workset

- Windows assigns a certain number of pageframes (dynamically adjusted) for each process, called the process workset
- Windows calculates the default maximum and minimum values of the workset based on the size of physical memory; Therefore, the default values for all processes are the same

Physical memory scale	The minimum number of pages frames by default for the workset	The maximum number of pageframes by default in the workset
Small (Less than19 MB)	20	45
Medium-sized (20 MB to 32 MB)	30	145
Large-scale (Larger than 32MB)	50	345

Windows page frame layout policy

- When a process experiences a pagination error, the memory manager checks the number of idle pageframes in the system
 - If conditions allow, the working set can be increased continuously to the maximum
 - If there are not enough idle frames, page replacement will be selected
- When the number of idle frames in memory becomes low, the system will "trim" the workset as needed to increase the available idle frames
- Windows dynamically checks the workset and lists the processes to be trimmed in order of priority

Hit rate and error cost

- To improve access efficiency, you must consider this :
 - **Hit rate** : The ratio of the number of times the required data is accessed on a specified device to the total number of accesses
 - **The Cost of Mistakes** : The required data is read from devices at the next layer of storage, replaced by devices at the previous layer, and then accessed by the CPU
- Regarding the requirements tab :
 - **Hit rate** : Number of times the desired page is accessed from memory / total number of accesses
 - **The Cost of Mistakes** : Replace a page in memory with the desired page on disk, plus the total time the CPU takes to access that page from memory

Calculate the average time to access data in the required tabs

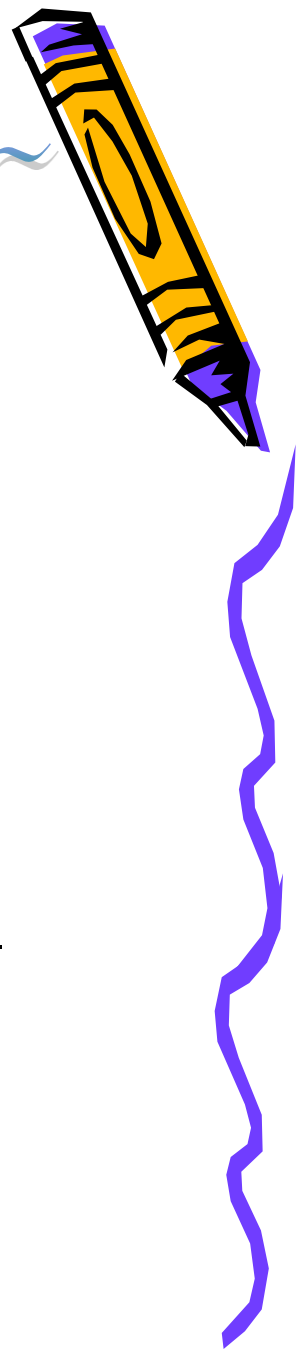
- Assume each time data is accessed from memory for M
- If there is no required data in memory and the error of reading a page from disk, the cost is D ($M \ll D$)
- The hit rate of the Demand tab is h
- Average time per data access:
 - $T = h * M + (1-h) * D$
- To reduce the average access time T , you can start by lowering the cost of errors D and increasing the hit rate h

Classroom exercises



- Assuming the time to access data from memory is 25ns, the error cost of reading pages from disk is 25ms
- If a page is accessed 1,000 times per time, it will fail on average once on average. May I ask?:
 - What is the hit rate of the Demand tab?
 - What is the average time for data access?
- 1. If the system switches to the new algorithm, it will fail three times per 4,000 page accesses on average. What are the hit rates and average times?





Practice Answers

1. Hit rate : 0.999

Average time :

$$25\text{ns} \times 0.999 + 25\text{ms} \times 0.001 = \underline{25025\text{ns}}$$

2. Hit rate : 0.99925 (1-3/4000)

Average time :

$$25\text{ns} \times 0.99925 + 25\text{ms} \times 0.00075 = \underline{18761\text{ns}}$$



The impact of program structure on pagination error frequency

- The way the program is designed can sometimes affect the frequency of paging errors
- Take array access as an example :
 - Assume each page is 256 bytes in size, each integer is 4 bytes long, and the integer array is stored in column priority `int N[64][64]`
 - Suppose during execution, there is only one page frame to store array data

```
for (j=0; j<64; j++)  
    for (i=0; i<64; i++)  
        N[i][j] = 0;
```

```
for (i=0; i<64; i++)  
    for (j=0; j<64; j++)  
        N[i][j] = 0;
```

There were a total of $64 * 64$ pagination errors There were a total of 64 pagination errors

5-4 Pagination Tabs

- Modern computer systems have logical addresses of **32 bits or more**, so the total length of the entire page break table is also quite large
- Problems caused by the large tab tabs :
 - 1.How can you find enough memory space to store a page spreadsheet? Should you store it in a continuous space, or simply split it into pages?
 - 2.If you want to split a page list, how can you quickly find the page information you need to access in a large and scattered page list?

Two-level pagination table

- The page break table itself is also divided into pages the same size as the memory pageframe, and each is placed into the available pageframes
- A separate table must be created to record the page positions of the page break, thus forming a two-layer page table
- A new table that records the page positions of these page spreads is called an outer page tab, root page tab, or page directory
- In a two-level paged table structure, accessing the required memory address requires at least two memory accesses

Figure 5-16 Two-level pagination

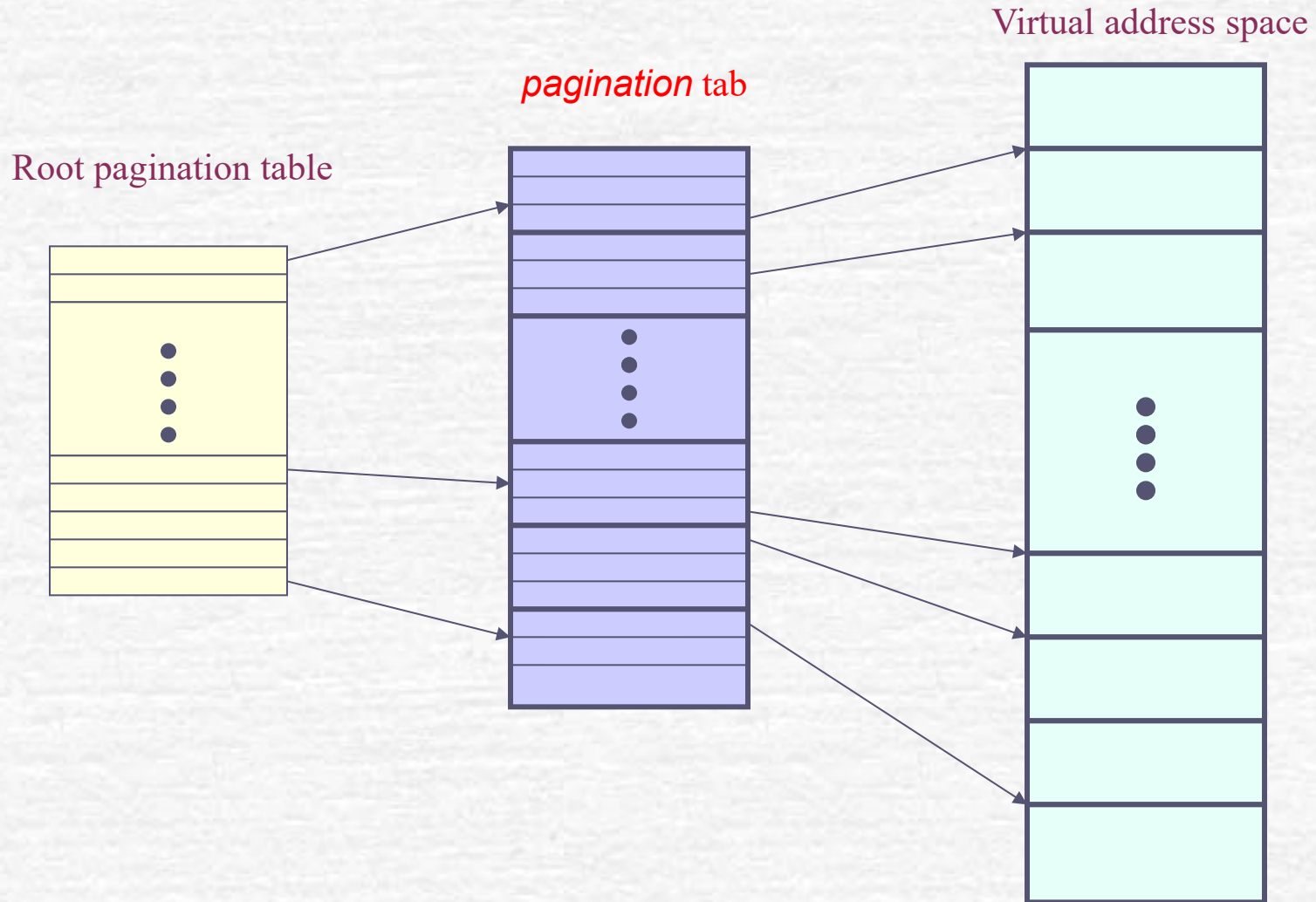
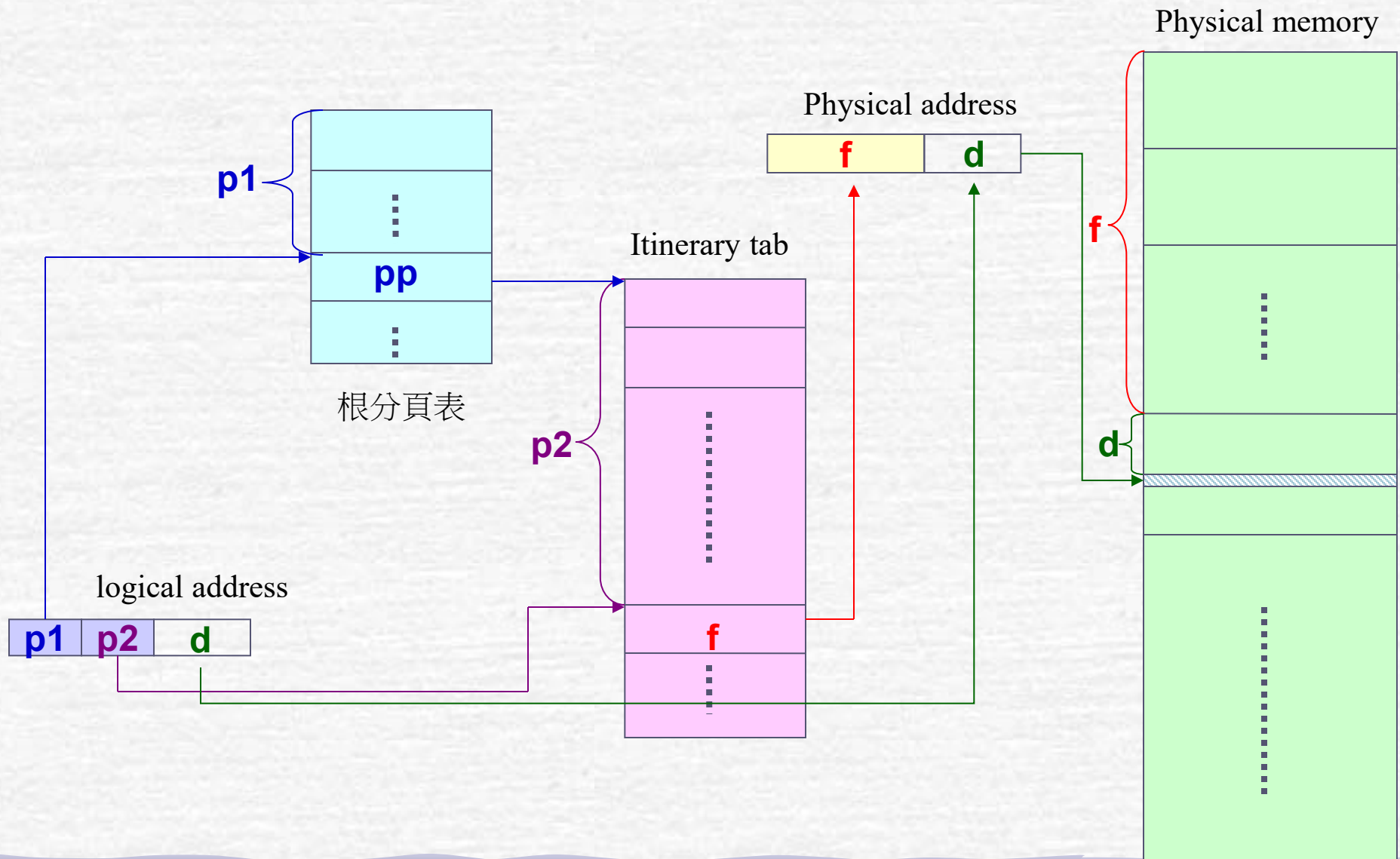


Figure 5-17 Address conversion in a two-level page table



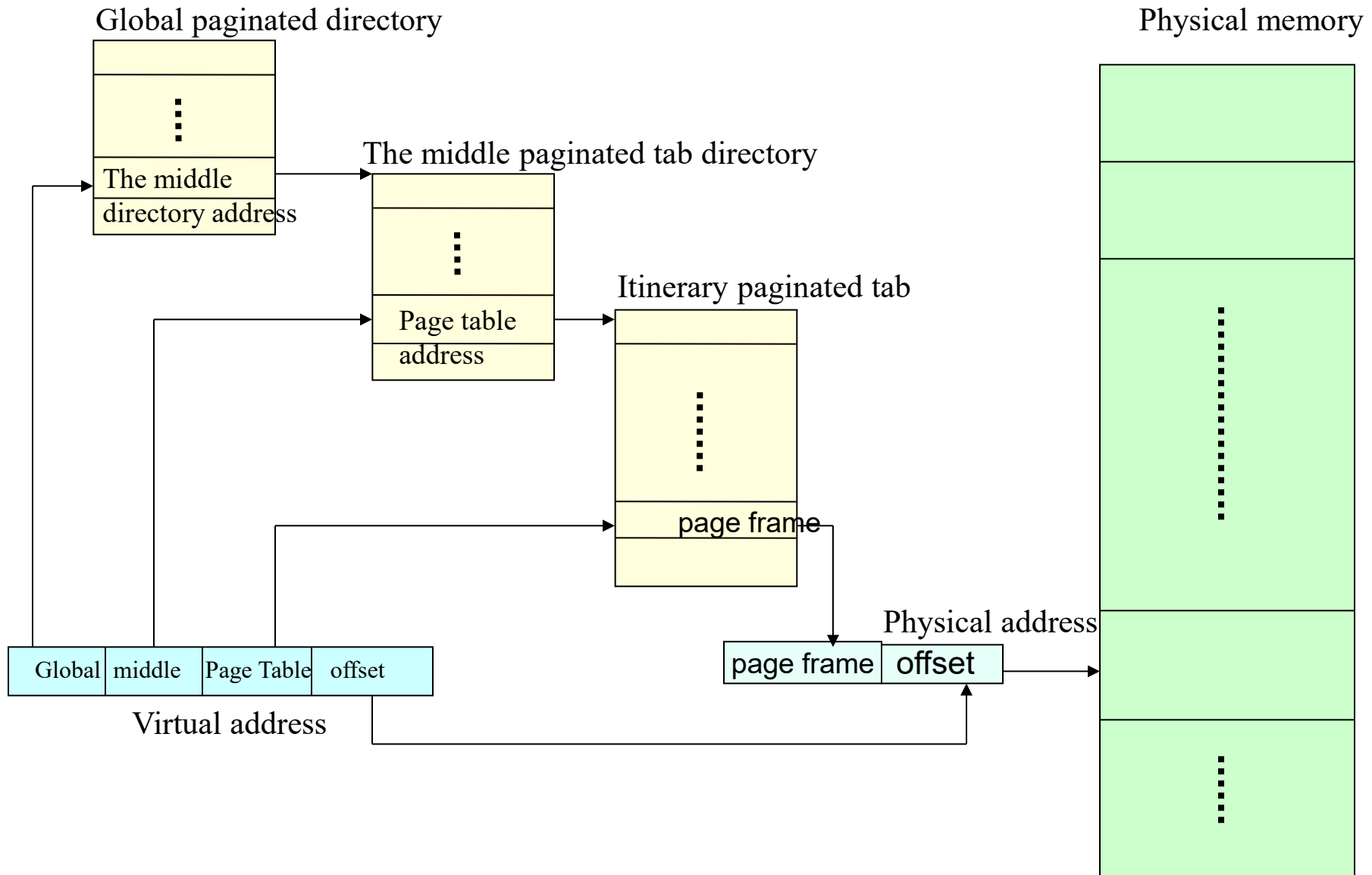
Multi-level page spread

- When the logical address space is extremely large, the outer page breaks may become too large, requiring further segmentation, which can lead to multi-layer structures such as three-layered or four-layered structures
- When hierarchical pagination has **too many layers**, the number of physical memory accesses required per memory reference also increases, which can **seriously impact performance**
- For **an n-level architecture**, at least n accesses are required

Implementation Discussion -- Linux Three-Tier Pagination

- Linux uses a **three-tier pagination system**: the global pagination directory, the intermediate pagination directory, and the pagination table, so Linux can be used for 64-bit architectures
- Linux logical addresses are divided into four parts, each serving as an index for different levels of page tables
- When Linux is applied in a 32-bit architecture, it does not use a middle page directory, but this part is still retained so that the same code can run on either 32-bit or 64-bit architectures

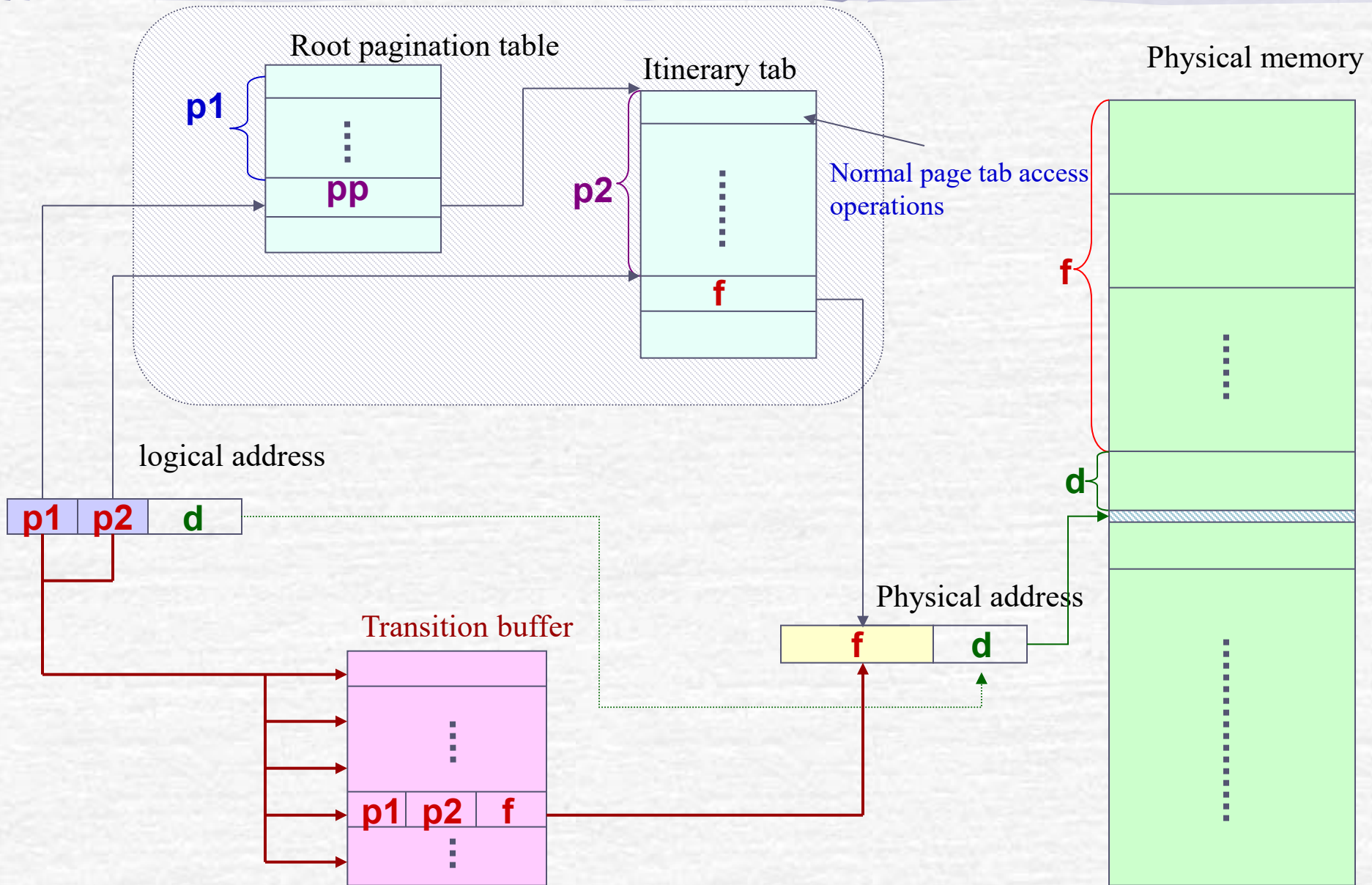
Figure 5-18 Linux pagination pattern



How to improve the performance of page spreads

- When the page break table is very large, commonly used page table items can be placed in a specially designed **Transformation Buffer (TLB)**.
- This specially designed high-speed memory is called associative memory, which can read all memory cells simultaneously and compare them with target values, performing parallel searches for stored data items
- When a logical address needs to be referenced, the processor simultaneously checks the TLB while **accessing the normal page table**

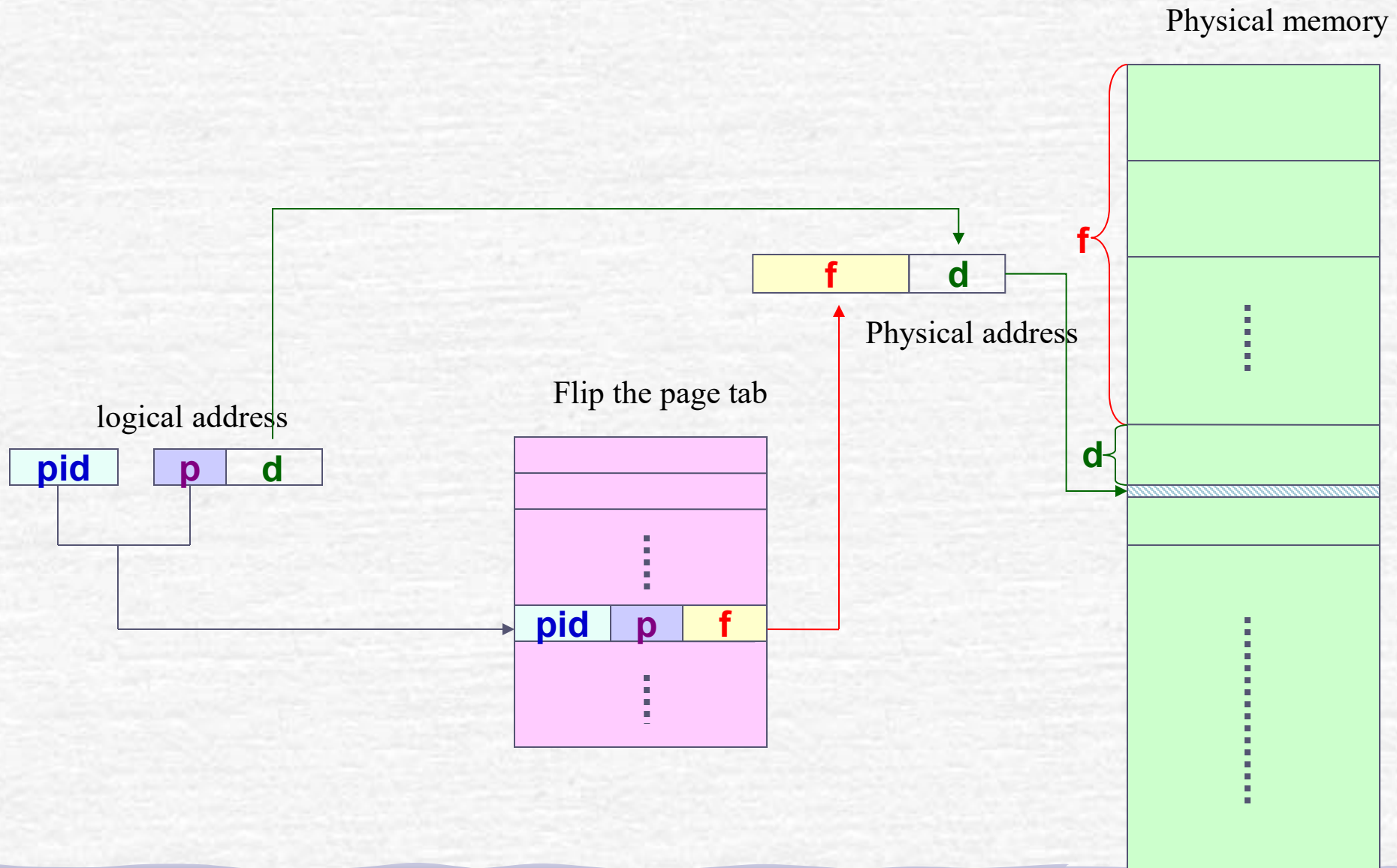
Figure 5-19 Schematic diagram of associated memory



Flip the page tab

- If the number of available pageframes is far less than the total logical address space of each process, creating a table to record the current content of each pageframe saves much more space than recording page data for each process
- The number of entries in the flipped page table equals the total number of pages in memory
- When a process wants to access a logical address, it can first check whether the required page is present in the table
 - If it exists, it means the page is already stored in physical memory, and you can directly retrieve its corresponding page frame number to calculate the physical address
 - Conversely, it indicates a pagination error, and the required page must be moved in before actual read/write operations can be performed

Figure 5-20 Address conversion when flipping a page table



Implementation issues with flipping page tables

- Each process may have the same page number, so in the flipped page table, in addition to storing the **logical page number**, you must also store the process number to which the page belongs to avoid **confusion**
- How to quickly find the required page item or make sure the page is not currently in physical memory: using **hash table techniques**

Figure 5-21 Inverted page table using hash method

